AWS MLA-C02 • DOMAIN 4 • PART 2

Optimizing Cost and Performance for ML and AI Infrastructure

Operating, Monitoring, and Securing ML and AI Solutions

02

THEORY-ONLY LEARNING SERIES

Monitor • Optimize • Secure



Choose the right instance family

Match the hardware to the model, not the other way around.

CPU

General-purpose and compute-optimized instances.

- ml.c and ml.m families
- Classical ML models
- Low cost per hour

GPU

NVIDIA GPU instances for heavy deep learning.

- ml.g5, ml.g6 inference
- ml.p4, ml.p5 large jobs
- Large models and LLMs

AWS Inferentia

AWS-designed chips built for inference.

- ml.inf2 instances
- Low cost per inference
- Uses the Neuron SDK

AWS Graviton

Arm-based CPUs with strong price-performance.

- ml.c7g and similar
- CPU inference savings
- Check ARM support



Find the bottleneck before you scale

Adding instances does not fix a slow step somewhere else.

Amazon CloudWatch

Endpoint and instance metrics show where time goes.

- CPU and GPU utilization
- Memory utilization
- ModelLatency vs overhead

AWS X-Ray

Traces one request across every service it touches.

- API Gateway to Lambda
- Lambda to the endpoint
- Finds the slow hop

AgentCore Observability

Breaks agent requests into timed steps.

- Slow tools or lookups
- Tokens per step
- Retries and loops



Dashboards serve two different audiences

Operators need live health; leaders need trends and cost.

CloudWatch dashboards

- Near real-time operational view
- Latency, errors, and utilization
- Alarm status in one place
- Built for engineers on call

Amazon QuickSight

- Business and cost trends over time
- Cost and Usage Reports via Athena
- Interactive filters and sharing
- Built for managers and stakeholders



Plan capacity for latency, reliability, and cost

Right-size first, then scale, then build in resilience.

Right-size

Pick the smallest instance that meets the target.

- Inference Recommender
- Load test real payloads
- Review every quarter

Scale smart

Let auto scaling follow real demand.

- Set min and max counts
- Tune cooldown periods
- Scale to zero when idle

Build resilience

Avoid a single point of failure.

- Two or more instances
- Spread across AZs
- Quotas sized for peaks



Control and reduce ML spend

See costs clearly, set limits, then buy capacity wisely.

Visibility

Cost Explorer and cost allocation tags.

- Cost by service and tag
- Tag jobs and endpoints
- Forecast future spend

Limits

AWS Budgets and service quotas set guardrails.

- Budget alerts
- Budget actions
- Quotas cap usage

Purchase options

Pay less for predictable or flexible usage.

- SageMaker Savings Plans
- Spot for training
- Provisioned Throughput



Foundation model costs are driven by tokens

Every prompt, response, and embedding has a price.

Token costs

Input and output tokens are priced separately.

- Shorter prompts
- Set max output tokens
- Track tokens in CloudWatch

Cost levers

Bedrock features that cut cost per request.

- Prompt caching
- Batch inference
- Smaller or distilled models

RAG and agents

Hidden costs beyond the main model call.

- Embedding tokens at ingest
- Vector store storage
- Agent steps add calls



A cost and performance checklist

Fast and affordable systems are measured, not guessed.

1. Pick CPU, GPU, Inferentia, or Graviton per workload.
2. Use CloudWatch and X-Ray to find the real bottleneck.
3. Build CloudWatch and QuickSight dashboards.
4. Right-size, auto scale, and spread across AZs.
5. Tag resources, set budgets, and use Savings Plans or Spot.
6. Manage tokens, embeddings, and vector storage costs.

